

Despliegue del sitio Petrogas 2026

Manual breve para poner en marcha el sitio nuevo de **petrogassa.com** en el VPS. Son dos contenedores y una base de datos, todo con `docker compose`.

No hace falta conocer la aplicación: cada paso dice qué tiene que verse si salió bien.

Qué se instala

Contenedor	Qué es	Puerto en el host
backend	API en NestJS	127.0.0.1:3100
frontend	Sitio en React, servido por nginx	127.0.0.1:3101
db	PostgreSQL 17	ninguno (solo red interna)

Los tres escuchan **solo en loopback**. El nginx del host es el único que les habla, y es quien sigue terminando el TLS como hasta ahora.

Requisitos

- Docker Engine con el plugin `compose`.
- El usuario que despliega tiene que poder correr `docker` (grupo `docker` o Docker rootless).
- Unos 3 GB de disco y 2 GB de memoria libre durante la compilación.

Comprobación:

```
docker compose version
```

1. El código

Los dos proyectos van **uno al lado del otro**. Esa disposición no es decorativa: el `docker-compose.prod.yml` del backend construye el frontend usando `../frontend` como contexto.

```
/var/www/petrogassa.com/app-2026/  
+-- backend/      <- repositorio del backend  
+-- frontend/     <- repositorio del frontend  
\-- data/         <- se crea en el paso 3 (base de datos e imágenes)
```

```
mkdir -p /var/www/petrogassa.com/app-2026
```

Después se clonan los dos repositorios dentro de esa carpeta, cada uno en el nombre que indica el esquema de arriba.

El sitio actual vive en `/var/www/petrogassa.com/public` y **no se toca**. Todo esto queda fuera de esa carpeta, así que no es accesible por web y el sitio en producción sigue funcionando igual mientras tanto.

2. El archivo de configuración

El archivo `.env.prod` va dentro de `backend/`. Se entrega aparte porque contiene credenciales y no viaja en el repositorio.

Antes del primer arranque hay que revisar dos valores:

- `**GESTION_API_TOKEN` — **viene vacío. Es el token de la API de Gestión, sin él la aplicación no arranca**** (falla con un mensaje claro).
- `**DATABASE_PASSWORD` — **conviene cambiarla ahora. PostgreSQL solo la aplica al crear la base la primera vez: cambiarla después no tiene efecto**** y hay que borrar la carpeta de datos para que vuelva a tomarla.

El resto de los valores ya está listo y no necesita ajustes.

3. La carpeta de datos

```
mkdir -p /var/www/petrogassa.com/app-2026/data/storage
```

Ahí van las imágenes que se suben desde el panel de administración.

Hay que crearla **a mano y antes** de levantar. Si la crea Docker queda como `root`, y el contenedor corre como usuario no-root: no podría escribir, y las subidas de imágenes fallarían.

Esa misma carpeta `data/` es la que hay que incluir en las copias de seguridad: contiene la base de datos y todas las imágenes del sitio.

4. Levantar

Desde `backend/`:

```
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --build
```

La primera vez tarda varios minutos: compila las dos imágenes desde cero.

Durante la construcción del frontend aparece este aviso, y **es esperado**:

```
AVISO: no se pudo leer la API (...)
Las páginas de detalle quedan sin vista previa al compartir.
```

Ocurre porque el frontend consulta la API pública durante la compilación, y en este momento el dominio todavía apunta al sitio anterior. Se resuelve solo en el paso 7. El sitio funciona igual.

5. Comprobar que arrancó

```
docker compose -f docker-compose.prod.yml --env-file .env.prod ps
```

Los tres servicios tienen que figurar como `running`, y `backend` y `db` como `healthy`.

Después, el registro del backend:

```
docker compose -f docker-compose.prod.yml --env-file .env.prod logs backend
```

Tiene que aparecer, en este orden:

1. Varias líneas `Migration ... has been executed successfully` (son diez).

2. Servicios iniciales sembrados: 2
3. Certificaciones iniciales sembradas: 3
4. Nest application successfully started

Los puntos 2 y 3 solo salen en el primer arranque: cargan el contenido inicial del sitio. Si no aparecen y la base está vacía, algo falló y conviene avisar antes de seguir.

Y una respuesta de cada contenedor:

```
curl -s -o /dev/null -w 'API %{http_code}\n' http://127.0.0.1:3100/api
```

```
curl -s -o /dev/null -w 'SITIO %{http_code}\n' http://127.0.0.1:3101/
```

Los dos tienen que responder 200.

6. nginx

Falta que el dominio llegue a los contenedores. La aplicación necesita que se repartan así:

Ruta	A dónde
/api/...	backend, 127.0.0.1:3100
/media/...	backend, 127.0.0.1:3100 (imágenes subidas)
/sitemap.xml	backend, 127.0.0.1:3100 (se genera solo)
todo lo demás	frontend, 127.0.0.1:3101

Bloque de referencia, para sumar al `server` que ya atiende `petrogassa.com` en el 443:

```
# Subidas del panel: el archivo más grande admitido son 10 MB.
client_max_body_size 12M;

location /api/ {
    proxy_pass http://127.0.0.1:3100;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    # El envío de un CV viaja a otra API y puede tardar.
    proxy_read_timeout 120s;
}

location /media/ {
    proxy_pass http://127.0.0.1:3100;
    proxy_set_header Host $host;
}

location = /sitemap.xml {
    proxy_pass http://127.0.0.1:3100;
    proxy_set_header Host $host;
}

location / {
    proxy_pass http://127.0.0.1:3101;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
```

```

    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

```

Las cabeceras X-Forwarded-* no son de adorno: la aplicación las usa para conocer la IP real de cada visitante, que es con lo que limita los intentos de inicio de sesión. Sin ellas, todos los visitantes comparten un mismo cupo.

El certificado actual ya cubre el dominio y no hay que emitir nada nuevo.

7. Reconstruir el frontend

Una vez que el dominio apunta al sitio nuevo, una última reconstrucción:

```
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --build frontend
```

Ahora sí la API responde en el dominio público, y el aviso del paso 4 desaparece. Esto es lo que genera las vistas previas con foto y título cuando alguien comparte una página del sitio por LinkedIn o WhatsApp.

Operación

Ver los registros en vivo

```
docker compose -f docker-compose.prod.yml --env-file .env.prod logs -f backend
```

Actualizar a una versión nueva

```
git pull && docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --build
```

Las migraciones de la base corren solas al arrancar y no se repiten.

Detener

```
docker compose -f docker-compose.prod.yml --env-file .env.prod down
```

down no borra nada: la base y las imágenes viven en data/, fuera de los contenedores.

Si algo falla

Qué se ve	Qué es
El backend reinicia en bucle y el log dice <code>GESTION_API_TOKEN</code>	Falta ese valor en <code>.env.prod</code> (paso 2)
Fallo al sembrar el contenido inicial	La base no estaba lista; reiniciar el backend alcanza
El panel no puede subir imágenes	La carpeta <code>data/storage</code> no es escribible (paso 3)
El sitio carga pero la API da error	El nginx del host no está mandando <code>/api</code> al 3100
Las imágenes del sitio no se ven	Falta la regla de <code>/media</code> en nginx

El arranque nunca falla en silencio: si una variable de configuración está mal, el contenedor se detiene y el motivo queda escrito en el registro.